

CAF — Operations & Deploy

Aggregated export — 2026-06-16. Source markdown in repo is unchanged.

Bundle id: `09-caf-operations-and-deploy`

Included: docs/FLY_PRODUCTION_CHECKLIST.md

Goal

Reduce time-to-complete and eliminate common Fly.io failure modes (timeouts, cold starts, wrong base URLs, renderer/video-assembly hangs) for **CAF Core** + **CAF Renderer/Media Gateway** deployments.

Biggest latency contributors (where to look)

- **LLM generation (OpenAI chat):** `src/services/openai-chat.ts` , `src/services/llm-generator.ts`
- **Carousel rendering** (per-slide Puppeteer): `services/renderer/server.js` + Core caller in `src/services/job-pipeline.ts` (`POST /render-binary` per slide)
- **Scene video assembly:**
 - **Sora polling:** `src/services/sora-scene-clips.ts` (OpenAI Videos API polling + download/upload)
 - **HeyGen polling:** `src/services/heygen-renderer.ts`
 - **Concat/mux:** `services/video-assembly/server.js` (ffmpeg)

Fly deployment shapes (pick one)

Option A (recommended): combined gateway on one Fly app

Deploy `services/media-gateway` image (spawns renderer + video-assembly on localhost).

- **Pros:** one stable public base URL for both render + video endpoints; avoids “wrong base URL” errors; easiest for `RENDERER_BASE_URL / VIDEO_ASSEMBLY_BASE_URL` .
 - **Deploy:** `fly deploy -c fly.caf-renderer.toml`
 - **Public endpoints:**
-

- **Renderer:** `POST /render-binary` , `GET /renderer/ready` , `GET /renderer/health` , `GET /templates/*`
- **Video assembly:** `POST /concat-videos` , `POST /mux` , `GET /status/:id` , `POST /full-pipeline`
- **Gateway readiness:** `GET /ready`

Option B: standalone services (only if you must)

- **services/renderer** Fly app: Puppeteer only.
- **services/video-assembly** Fly app: ffmpeg only.
- **Downside:** you must set two different base URLs and ensure the "status polling" endpoint is exposed on the same host you use for async jobs.

Required env vars (exact)

On **caf-core** (Fly app **caf-core**)

- **RENDERER_BASE_URL** :
 - If using combined gateway: `https://caf-renderer.fly.dev`
 - If using standalone renderer: `https://<your-renderer-app>.fly.dev`
- **VIDEO_ASSEMBLY_BASE_URL** :
 - If using combined gateway: `https://caf-renderer.fly.dev`
 - If using standalone video-assembly: `https://<your-video-assembly-app>.fly.dev`

Constraints / gotchas:

- **No trailing slash** is safest (Core trims it, but keep it consistent).
- **VIDEO_ASSEMBLY_BASE_URL** must serve **both** `POST /concat-videos?async=1` and `GET /status/:requestId` . If you point it at a host that only proxies the POST routes, Core will fail during polling.

On **caf-renderer** (combined gateway app)

- **Recommended:**
 - `CAF_TEMPLATE_API_URL=https://caf-core.fly.dev` (so renderer can fetch missing templates from Core)
 - `SUPABASE_URL` , `SUPABASE_SERVICE_ROLE_KEY` , `SUPABASE_ASSETS_BUCKET` (so video-assembly can upload outputs and return `public_url`)
- **Timeout hardening** (safe production defaults):

- VIDEO_ASSEMBLY_FETCH_TIMEOUT_MS=180000
- VIDEO_ASSEMBLY_FFMPEG_TIMEOUT_MS=1200000
- VIDEO_ASSEMBLY_JOB_TIMEOUT_MS=1800000

OpenAI timeout hardening (CAF Core)

- OPENAI_CHAT_TIMEOUT_MS=180000
- OPENAI_CHAT_MAX_RETRIES=2

Notes:

- These are used by `src/services/openai-chat.ts` and `src/services/openai-chat-multimodal.ts` to prevent “hang forever” calls.

Fly health checks (readiness vs liveness)

- **caf-core**: Fly should check `GET /readyz` (DB connectivity)
- **caf-renderer (combined gateway)**: Fly should check `GET /ready` (Chromium launch + ffmpeg present)

Observability (minimal, production-safe)

- **Video-assembly** logs durations for `stitch`, `concat`, `mux`, and full-pipeline runs.
- **OpenAI chat** emits a console log when a call takes ($\geq$ 20s), including attempt number and tokens.
- **Core video render** emits a console log when the overall render step takes ($\geq$ 20s).

Fly smoke test plan

1) Readiness

- `curl https://caf-core.fly.dev/readyz`
- `curl https://caf-renderer.fly.dev/ready`

2) Dependency wiring sanity

- `curl https://caf-core.fly.dev/health/rendering`
 - Confirm `rendering.renderer.ok=true`
 - Confirm `rendering.video_assembly.ok=true`

3) Minimal renderer test

- `curl -X POST https://caf-renderer.fly.dev/render-binary -H "content-type: application/json" -d '{"template": "carousel_sns_chat_story", "data": {"render": {"slides": [{"headline": "Hello", "body": "World"}]}}' --output out.png`

4) Minimal video-assembly async test (concat)

- Start:
 - `curl -X POST "https://caf-renderer.fly.dev/concat-videos?async=1" -H "content-type: application/json" -d '{"video_urls": ["<public-mp4-1>", "<public-mp4-2>"]}'`
- Poll:
 - `curl https://caf-renderer.fly.dev/status/<request_id>`

5) End-to-end Core pipeline (small)

- Create a run with 1 candidate and process it (or use existing admin helpers).
- Watch logs:
 - Core logs for OpenAI timeouts/retries and overall video render durations.
 - Renderer/gateway logs for /ready readiness and any puppeteer errors.
 - Video-assembly logs for ffmpeg duration and any watchdog timeouts.

Included: docs/USER_INPUT_AND_SECRETS.md

Everything you need to provide (save this checklist)

Use this as your **single vault checklist**. Copy values only into `.env`, password managers, or deployment secret stores (Vercel / Fly) — never into git.

For variable names and comments, see also [.env.example](#) and [ENV_AND_SECRETS_INVENTORY.md](#).

1. CAF Core API (this service)

Item	Required?	Where it goes	Notes

Postgres DATABASE_URL	Yes	.env / Fly secret	Supabase pooler URI, Neon, RDS, or local Docker (see docker-compose.yml).
CAF_CORE_API_TOKEN	Recommended prod	.env / Fly secret	Long random string; pair with CAF_CORE_REQUIRE_AUTH=1 .
PORT / HOST	Optional	.env	Defaults 3847 / 0.0.0.0.
OPENAI_API_KEY / OPENAI_MODEL	Yes for generation	.env / Fly secret	Required for LLM generation, scene clips (Sora), TTS.
HEYGEN_API_KEY / HEYGEN_API_BASE	Yes for HeyGen video	.env / Fly secret	Only if using HeyGen avatar / video-agent flows.
SUPABASE_URL / SUPABASE_SERVICE_ROLE_KEY / SUPABASE_ASSETS_BUCKET	Yes for asset uploads	.env / Fly secret	Where render outputs (carousel PNGs, video MP4s, scene clips) are stored.
RENDERER_BASE_URL / VIDEO_ASSEMBLY_BASE_URL	Yes for rendering	.env / Fly secret	Point at services/renderer, services/video-assembly, or the combined media-gateway.
CAROUSEL_TEMPLATES_DIR	Optional	.env	Directory served via GET /api/templates/* for the renderer.
CAF_PUBLISH_EXECUTOR	Optional	.env	none (default) / dry_run / meta. For meta, add page tokens + META_GRAPH_API_VERSION (or per-project rows in project_integrations).
CAF_OUTPUT_SCHEMA_VALIDATION_MODE	Optional	.env	skip / warn / enforce; legacy CAF_SKIP_OUTPUT_SCHEMA_VALIDATION is still honored as a fallback.
Scoring & caps (SCORE_WEIGHT_*, DEFAULT_MIN_SCORE_TO_GENERATE, ...)	Optional	.env	Tuning only. See src/config.ts.

Migrations:

- Applied via **npm run migrate** , or automatically on API startup when **CAF_RUN_MIGRATIONS_ON_START** is true (default).
- Tracked in **caf_core.schema_migrations** .
- If you already applied migrations before the ledger existed, baseline it once:

```
INSERT INTO caf_core.schema_migrations (filename) VALUES
('001_caf_core_schema.sql')
ON CONFLICT DO NOTHING;
```

2. Review app (apps/review — Next.js 14)

The Review workbench is a **pure client of CAF Core**. It does not own its own database, queue, or third-party credentials — everything it shows comes from Core `/v1/*` routes.

Item	Required?	Notes
CAF_CORE_URL	Yes	Public URL of your Core deployment (e.g. <code>https://caf-core.fly.dev</code>); <code>http://localhost:3847</code> for local dev.
CAF_CORE_TOKEN	Yes when Core has <code>CAF_CORE_REQUIRE_AUTH=1</code>	Must match <code>CAF_CORE_API_TOKEN</code> on Core. Sent as <code>x-caf-core-token</code> / <code>Authorization: Bearer</code> .
REVIEW_WRITE_TOKEN	Recommended prod	Gate decision writes from the UI; leave empty for local dev.
NEXT_PUBLIC_APP_URL	Recommended	Used for preview/link building.
PROJECT_SLUG	Optional	Empty = load all active projects' queues. Set (e.g. <code>SNS</code> , <code>connectr</code>) to lock the workbench to one tenant.
REVIEW_ALL_PROJECTS	Optional	<code>1</code> / <code>true</code> forces cross-project queue even when <code>PROJECT_SLUG</code> is set.
REVIEW_FALLBACK_PROJECT_SLUG	Optional	Used only when Core lacks <code>/v1/review-queue-all/*</code> (older deploy). Default fallback is <code>SNS</code> .

All Core-dependent configuration (HeyGen, Supabase, Meta publishing, Google/OAuth credentials) stays on **Core** and `project_integrations` — not on the Review app.

3. Media services on Fly.io

Item	Notes
Fly app names	e.g. <code>caf-core</code> , <code>media-gateway</code> (renderer + video-assembly).

<code>FLY_API_TOKEN</code>	GitHub Actions secret for the deploy workflow.
<code>SUPABASE_URL</code> + <code>SUPABASE_SERVICE_ROLE_KEY</code> on Fly	Same Supabase project as Core uploads.
<code>OPENAI_API_KEY</code> on Fly (if needed)	Only when the video-assembly uses Whisper / mux alignment.
<code>CAF_TEMPLATE_API_URL</code>	Core base URL, so the renderer can fetch .hbs templates via <code>GET /api/templates/*</code> .
Renderer / mux tuning	Timeouts, burn style, retries — listed in <code>.env.example</code> .

See also [FLY_PRODUCTION_CHECKLIST.md](#) .

4. Third-party provider accounts (vault)

Provider	Typical secret	Used by
OpenAI	API key (+ org / project id if scoped)	LLM generation, Sora scene clips, TTS, approval-review vision
HeyGen	API key / account	Avatar and video-agent renders
Supabase	Service-role key	Asset storage bucket
Meta Graph	Page tokens (per <code>project_integrations</code>)	Publishing when <code>CAF_PUBLISH_EXECUTOR=meta</code>

Per-project overrides (e.g. separate Meta tokens per brand) live in `caf_core.project_integrations` , not in env.

5. External publish workers (optional)

If you run a downstream worker to actually post content, it consumes Core's stable publish payload:

- **Endpoint:** `GET /v1/publications/:project_slug/:id/n8n-payload` (route name is historical — the shape is the Core-owned contract for any external publisher).
- **Auth:** `x-caf-core-token` / `Authorization: Bearer` when `CAF_CORE_REQUIRE_AUTH=1` .
- **Outcomes:** workers call back into `POST /v1/publications/.../complete` (or patch the placement) to record `published` / `failed` state.

Nothing in Core requires a specific external orchestrator; the publish executor is selectable via `CAF_PUBLISH_EXECUTOR` (`none` / `dry_run` / `meta`).

6. End-state "done when" checklist

- ☐ `DATABASE_URL` set; `npm run migrate` succeeds; `npm run dev` responds on `/health` .
- ☐ If auth on: `CAF_CORE_REQUIRE_AUTH=1` + `CAF_CORE_API_TOKEN` ; every client (Review app, scripts, workers) sends the matching token.
- ☐ Review app deployed with `CAF_CORE_URL` (public) + optional `CAF_CORE_TOKEN` ; review queue loads.
- ☐ Fly apps deployed; Core's `RENDERER_BASE_URL` / `VIDEO_ASSEMBLY_BASE_URL` point at the deployed media services.
- ☐ `OPENAI_API_KEY` , `HEYGEN_API_KEY` , and `SUPABASE_*` set wherever they are needed (Core, and on Fly for the media stack that uploads assets).
- ☐ `CAF_PUBLISH_EXECUTOR` set explicitly for production (`meta` with project-integration tokens, or `none` / `dry_run` while testing).
- ☐ No secrets committed; `.env` is git-ignored.

This file is safe to commit. It contains no secret values.

Included: ENV_AND_SECRETS_INVENTORY.md

Environment variables and secrets inventory (CAF)

Purpose: One checklist of **every** configuration name used by this repo and adjacent ops (n8n, Fly, Vercel), where to set it, and whether it is secret.

External readers: See also `docs/EXTERNAL_CONTEXT_PACK.md` (Tier 1 includes this file + `.env.example`) and `docs/REBUILD_FROM_DOCS.md` (minimum env for bootstrap).

Security — non-negotiable

- **Never** commit real tokens, keys, JSON service accounts, or refresh tokens to git.
 - Copy `.env.example` → `.env` locally (`.env` is gitignored).
 - Store production values in **Vercel / Fly secrets / n8n credentials / your vault** only.
 - The “**Your value (vault only)**” column below is for you to fill **outside** the repo.
-

Legend

Column	Meaning
Secret?	yes = treat like a password; no = non-sensitive URL or flag; optional secret = only if you use that feature
Where to set	Typical location for production

0. CAF Core API (repo root) — `src/config.ts`

Canonical list: every variable is defined in `src/config.ts` (Zod) with defaults. Comments in `.env.example`.

Variable	Secret?	Required	Purpose
DATABASE_URL	yes	yes	PostgreSQL connection (caf_core schema)
OPENAI_API_KEY	yes	for LLM	Generation, QC helpers, mimic copy, insights
OPENAI_MODEL	no	recommended	Default chat model
PORT / HOST	no	optional	API listen (default 3847)
CAF_CORE_REQUIRE_AUTH	no	optional	1 to require token on protected routes
CAF_CORE_API_TOKEN	yes	when auth on	x-caf-core-token Or Authorization: Bearer
CAF_RUN_MIGRATIONS_ON_START	no	optional	Apply migrations on startup (default on)
RENDERER_BASE_URL	no	for carousel	Carousel worker URL
VIDEO_ASSEMBLY_BASE_URL	no	for video	ffmpeg worker URL
SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY, SUPABASE_ASSETS_BUCKET	yes / no	for assets	Object storage uploads
HEYGEN_API_KEY	yes	for HeyGen flows	Video agent / avatar renders
MIMIC_IMAGE_ENABLED	no	optional	Master switch for mimic render
MIMIC_IMAGE_PROVIDER	no	optional	bfl dashscope nvidia openai

BFL_API_KEY, DASHSCOPE_API_KEY, NVIDIA_NIM_API_KEY	yes	per provider	Mimic image edit providers
CAF_PUBLISH_EXECUTOR	no	optional	none dry_run meta
CAF_REQUIRE_HUMAN_REVIEW_AFTER_QC	no	optional	Human gate after QC (default true)
CAF_OUTPUT_SCHEMA_VALIDATION_MODE	no	optional	skip warn enforce

Mimic-specific vars: **\$10** below. Full bootstrap: [docs/REBUILD_FROM_DOCS.md](#) .

1. Review app (apps/review) — Vercel / local

Review is a **Next.js client of CAF Core** — job state lives in Postgres via Core APIs, not in the Review app database.

Variable	Secret?	Required	Where to set	Used for
CAF_CORE_URL	no	yes	Vercel env, apps/review/.env.local	Core API base http://localhost
CAF_CORE_TOKEN	yes	when Core auth on	Vercel env	Sent to Core as core-token
CAF_CORE_API_TOKEN	yes	optional alias	Some routes	Alternate name by a few API routes
REVIEW_WRITE_TOKEN	yes	optional	Vercel env	Protects POST /api/task/.../ in Review
RENDERER_BASE_URL	no	recommended	Vercel env	Carousel preview proxies (default http://localhost)
NEXT_PUBLIC_APP_URL	no	recommended	Vercel env	Absolute links
PROJECT_SLUG	no	optional	Vercel env	Lock workbench one tenant; error all active projects
REVIEW_ALL_PROJECTS	no	optional	Vercel env	1/true = cross-queue even if PROJECT_SLUG S
REVIEW_FALLBACK_PROJECT_SLUG	no	optional	Vercel env	Fallback when Core lacks /v1, queue-all/*

Legacy (not used by current Core-backed Review): Google Sheets queue vars (`GOOGLE_REVIEW_QUEUE_*` , `GOOGLE_SERVICE_ACCOUNT_JSON` , etc.) may appear in old deploy notes — the current app reads the review queue from **Core /v1/ APIs**.

2. Renderer service (`services/renderer`) — Fly or local

Variable	Secret?	Required	Where to set	Used for	Your value (vault only)
PORT	no	optional	Fly, local	Listen (default 3333)	
CAF_TEMPLATE_API_URL	no	optional	Fly secret	Base URL of this Next app for Supabase-backed templates	
RENDERERS_BEFORE_RESET	no	optional	Fly	Puppeteer restarts after N renders (default 12)	
RENDERER_SHUTDOWN_SECRET	yes	optional	Fly secret	POST /shutdown protection	

3. Video assembly (`services/video-assembly`) — Fly or local

Variable	Secret?	Required	Where to set	Used for	Your value (vault only)
PORT	no	optional	Fly, local	Listen (default 3334)	
SUPABASE_URL or NEXT_PUBLIC_SUPABASE_URL	no	yes	Fly secret	Upload target project	
SUPABASE_SERVICE_ROLE_KEY	yes	yes	Fly secret	Storage + optional tasks update	
SUPABASE_ASSETS_BUCKET	no	optional	Fly secret	Bucket (default assets)	

OPENAI_API_KEY	yes	optional	Fly secret	Whisper alignment when requested	
OPENAI_WHISPER_MODEL	no	optional	Fly	Default whisper-1	
OPENAI_WHISPER_TIMEOUT_MS	no	optional	Fly	Whisper HTTP timeout	
VIDEO_ASSEMBLY_FETCH_TIMEOUT_MS	no	optional	Fly	Download clips/video	
VIDEO_ASSEMBLY_FFMPEG_TIMEOUT_MS	no	optional	Fly	ffmpeg processes	
VIDEO_ASSEMBLY_JOB_TIMEOUT_MS	no	optional	Fly	Whole stitch/mux job	
FFMPEG_PATH	no	optional	Fly	Override binary	
FFPROBE_PATH / FFPROBE_TIMEOUT_MS	no	optional	Fly	Probe media	
TMP_DIR	no	optional	Fly	Temp working dir	
MUX_BURN_SUBTITLE_FORCE_STYLE	no	optional	Fly	ASS force_style	
MUX_BURN_ENCODE_PRESET / MUX_BURN_ENCODE_CRF	no	optional	Fly	Burn-in encode	
MUX_CHUNK_SCRIPT_MAX_CHARS	no	optional	Fly / body	TTS chunking	
MUX_MIN_CAPTION_SEC	no	optional	Fly	Caption pacing	
MUX_CAPTION_WEIGHT_EXPONENT	no	optional	Fly	Caption pacing	
MUX_CAPTION_TIMING_MODE	no	optional	Fly	equal / weighted	
MUX_USE_SCENE_DURATION_FOR_SRT	no	optional	Fly	Legacy timing	
MUX_SUBTITLE_DURATION_SCALE	no	optional	Fly	Scale scene SRT	
MUX_SUBTITLE_ALIGN	no	optional	Fly	e.g. whisper	

MUX_WHISPER_CAPTION_SOURCE	no	optional	Fly	payload / transcript	
MUX_SYNC_SUPABASE_TASK	no	optional	Fly	0 to skip task row update	
SUPABASE_UPLOAD_MAX_ATTEMPTS / SUPABASE_UPLOAD_RETRY_MS	no	optional	Fly	Upload retry	

4. Media gateway (`services/media-gateway`) — Fly

Variable	Secret?	Required	Where to set	Used for	Your value (vault only)
PORT	no	optional	Fly	Gateway listen (default 8080)	
RENDERER_PORT	no	optional	Fly	Child renderer port	
VIDEO_ASSEMBLY_PORT	no	optional	Fly	Child video-assembly port	

Child processes inherit **same env** as gateway on Fly — set Supabase and mux vars on the **gateway app** so stitch/mux can upload.

5. GitHub Actions (deploy Fly media-gateway)

Name	Type	Secret?	Where to set	Used for	Your value (vault only)
FLY_API_TOKEN	repository Secret	yes	GitHub → Settings → Secrets	flyctl deploy (<code>.github/workflows/fly-renderer.yml</code>)	
FLY_RENDERER_APP	repository Variable	no	GitHub → Settings → Variables	Override Fly app name (-a)	

6. n8n (instance credentials — not in `.env.example`)

Store these in **n8n** → **Credentials**, not in the CAF repo.

Credential type	Typical use in CAF flows	Your value (vault only)
Google Sheets OAuth2	CREATION, VALIDATION, INPUT, PROCESSING, Signal Pack	
OpenAI API	LLM, TTS, Sora/video nodes	
Supabase	Direct DB/REST if any node uses it	
HeyGen	Video render	
HTTP Header Auth / Bearer	RENDERER_BASE_URL, VIDEO_ASSEMBLY_BASE_URL, mux async poll	
Custom	Apify, webhooks, etc.	

Also record:

Item	Your value (vault only)
n8n base URL	
n8n admin auth method	
Per-workflow webhook URLs (if used)	

7. Google Cloud / Sheets (operational inventory)

Fill in **outside** the repo (which spreadsheet belongs to which env).

Resource	Example / note	Your value (vault only)
GCP project for Sheets API		
OAuth consent / Desktop or Web client		
Service account email (if used)	Share each spreadsheet with Editor	
VALIDATION spreadsheet ID	Review Queue tab	
CREATION - Runtime spreadsheet ID	Signal_Packs, Content_Jobs, ...	
PROCESSING insights spreadsheet ID	Write Signal Pack reads from here	
Flow Engine spreadsheet ID	Global	
CREATION - Project Config per brand		

8. Other platforms (from 09_external_integrations.md)

Platform	What to store	Your value (vault only)
Supabase	Project ref, dashboard URL, service role key (above), anon key if any client use	
Fly.io	Org, app name(s) for gateway/renderer, region	
Vercel	Project name, team, env vars (mirror \$1)	
OpenAI	Org / project IDs if using scoped keys	
HeyGen	API key, account	
Apify	API token, actor IDs	

9. Quick “first deploy” checklist

- ☐ Supabase: project created; migrations or SQL applied; bucket `assets` public as needed
- ☐ Vercel: \$1 vars; `NEXT_PUBLIC_APP_URL` = production URL
- ☐ Google: Review Queue sheet shared with SA or OAuth account (Editor)
- ☐ Fly (optional): deploy media-gateway; set Supabase + mux + optional `CAF_TEMPLATE_API_URL`
- ☐ GitHub: `FLY_API_TOKEN` (+ optional `FLY_RENDERER_APP` variable)
- ☐ n8n: credentials + `RENDERER_BASE_URL` / `VIDEO_ASSEMBLY_BASE_URL` pointing at stable URLs
- ☐ Rotate any credential that ever touched a public repo

10. CAF Core API — top-performer mimic (`src/config.ts`)

Set on the **Core** host (repo root `.env` / Fly secrets). Copy generation requires

`OPENAI_API_KEY` even when render uses BFL/DashScope/NVIDIA.

Variable	Secret?	Default	Purpose
<code>MIMIC_IMAGE_ENABLED</code>	no	false	Master switch for mimic draft/render
<code>MIMIC_IMAGE_PROVIDER</code>	no	bfl	bfl dashscope nvidia openai
<code>BFL_API_KEY</code>	yes	—	BFL FLUX edits when provider=bfl

DASHSCOPE_API_KEY	yes	—	Alibaba Qwen edit when provider=dashscope
NVIDIA_NIM_API_KEY	yes	—	NIM Qwen edit when provider=nvidia
MIMIC_IMAGE_BFL_MODEL	no	flux-2-klein-4b	BFL model slug
MIMIC_VISUAL_SIMILARITY_PCT	no	70	Reference-edit fidelity hint
MIMIC_IMAGE_INPUT_MODE	no	reference_edit	reference_edit OR analysis_t2i
MIMIC_IMAGE_DEFAULT_SIZE	no	1024x1536	Output dimensions
LLM_MIMIC*_MAX_CHARS	no	see .env.example	Mimic prompt context caps

Full list: **.env.example** (comments) and **docs/MIMIC_FLOWS_COMPLETE_GUIDE.md** §13.

11. Duplicate reference

Canonical **comments** for many Next.js vars also live in **.env.example** at the repo root. Prefer updating **.env.example** when adding new app-level vars, and add a row here in the matching section.

*Inventory version: aligned with repo scan 2026-06-15. Update when new **process.env.*** appears in code.*